МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение высшего образования «Санкт-Петербургский политехнический университет Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 Математика и компьютерные науки

Методы тестирования программного обеспечения

Отчет по лабораторной работе №3 «Статическое тестирование»

Студент
группы 5130201/30101 _____ Шишмаков В.О.

Преподаватель _____ Курочкин М. А.

«____»_____ 2026 г.

Санкт-Петербург, 2026

Содержание

Введение **3**

1 Постановка задачи **4**

2 Описание тестируемой программы **5**

3 Статический анализ кода **7**

4 Описание приложения и среды разработки **8**

5 Запуск CppCheck на MacOS **12**

6 Результаты запуска **13**

7 Исправление недостатков **15**

Заключение **17**

Введение

Долгое время большинство программистов полагало, что программы создаются только для выполнения на машинах и не рассчитаны на чтение людьми, а единственным способом их проверки служит запуск кода на компьютере. Эта ситуация начала меняться в начале 1970-х годов благодаря тем разработчикам, которые первыми поняли ценность просмотра кода как части процесса тестирования и отладки.

Полезность вычитки кода определяется несколькими факторами: размер и сложность приложения, количество членов команды разработчиков, временные рамки проекта (например, мягкие или жёсткие), а также, безусловно, квалификация и профессиональная культура команды программистов.

Статический анализ кода представляет собой анализ исходного кода программного обеспечения, выполняемый без фактического запуска исследуемых программ.

Cppcheck — это инструмент статического анализа для языков C и C++, который предназначен для выявления проблем, не обнаруживаемых компиляторами.

1 Постановка задачи

Дано: программный код на языке программирования `C++` (Построение словаря на основе хэш-функции и красно-черного дерева. Кодирование данных).
Требуется: изучить метод статического анализа кода, провести статический анализ кода при помощи `CppCheck` версии 2.20.0.

2 Описание тестируемой программы

Название программы: нахождение кратчайшего пути между двумя вершинами графа методом обхода в ширину (BFS).

Дано:

- `n` — количество вершин графа;
- `oriented` — тип графа: ориентированный или неориентированный;
- `edges` — список рёбер графа, задаваемых пользователем вручную (пары вершин);
- `start` — начальная вершина пути;
- `end` — конечная вершина пути.

Требуется: найти кратчайший путь (по числу рёбер) от вершины `start` до вершины `end` в заданном графе методом обхода в ширину и вывести последовательность вершин этого пути.

Ограничения — все параметры целые числа:

- количество вершин $n \in [2; 10000]$
- тип графа `oriented` $\in \{y, n\}$
- начальная вершина `start` $\in [0; n-1]$
- конечная вершина `end` $\in [0; n-1]$, `end` $\neq$ `start`
- вершины рёбер $u, v \in [0; n-1]$, $u \neq v$

Спецификация

Входные данные	**Выходные данные**	**Реакция программы**
$n < 2$ или $n > 10000$	Сообщение: *«Incorrect input! Enter a number from 2 to 10000.»*	Повторный запрос ввода от пользователя до тех пор, пока не будет введено корректное значение.
`oriented` $\notin \{y, n, Y, N\}$	Сообщение: *«Incorrect input! Enter y or n.»*	Повторный запрос ввода от пользователя до тех пор, пока не будет введено корректное значение.

Входные данные	**Выходные данные**	**Реакция программы**
Ребро $u \notin [0;\, n{-}1]$ или $v \notin [0;\, n{-}1]$	Сообщение: *«Incorrect input!»*	Введённое ребро отклоняется; программа запрашивает следующее ребро.
Ребро $u = v$	Сообщение: *«Self-loops are not allowed!»*	Введённое ребро отклоняется; программа запрашивает следующее ребро.
Ввод -1 в поле «From» или «To»	Ввод рёбер завершается	Программа переходит к запросу начальной и конечной вершин.
`start` $\notin [0;\, n{-}1]$	Сообщение: *«Incorrect input! Enter a number from 0 to $n{-}1$.»*	Повторный запрос ввода до тех пор, пока не будет введено корректное значение.
`end` $\notin [0;\, n{-}1]$ или `end` $=$ `start`	Сообщение: *«Incorrect input! Enter a number from 0 to $n{-}1$ (not start).»*	Повторный запрос ввода до тех пор, пока не будет введено корректное значение.
Все параметры корректны; путь от `start` до `end` существует	Сообщение вида: *«Shortest path (k edges): $v_0 \to v_1 \to \cdots \to v_k$»*	Программа выполняет обход в ширину (BFS) по матрице смежности. Выводит последовательность вершин кратчайшего пути и число пройденных рёбер k. Завершает работу.
Все параметры корректны; путь от `start` до `end` **не существует**	Сообщение: *«No path from start to end.»*	Программа сообщает об отсутствии пути и завершает работу.

3 Статический анализ кода

Статический анализ кода проводится без фактического запуска программы. Он исследует исходный код для обнаружения возможных ошибок, уязвимостей и отклонений от принятых стандартов кодирования. Благодаря этому виду анализа разработчики могут находить проблемы на ранних этапах создания программы, что позволяет существенно снизить затраты на их исправление в дальнейшем.

Помимо прочего, статический анализ может включать проверку стиля оформления кода, что способствует поддержанию единообразия в проекте. Это особенно важно в командах, где над одним проектом трудится несколько разработчиков. Инструменты статического анализа можно настраивать на соответствие кода определённым стандартам — это помогает избегать типичных ошибок и повышает удобочитаемость кода.

Многие подобные инструменты можно встраивать в процесс сборки, что позволяет автоматизировать проверку кода. Такой подход обеспечивает непрерывный контроль качества и даёт разработчикам возможность больше сосредоточиться на написании кода, а не на его ручной проверке.

Виды статического тестирования

Статический анализ кода (Static Code Analysis) — этот вид статического тестирования подразумевает исследование исходного кода. Его главная цель — выявление потенциальных проблем, некорректных практик, структурных аномалий и нарушений стандартов кодирования. Такие инструменты, как Lint, Pylint и ESLint, помогают автоматизировать этот процесс.

Обзоры кода (Code Reviews) — данный вид статического тестирования заключается в анализе кода членами команды разработки или приглашёнными экспертами. Обзоры кода позволяют находить проблемы и несоответствия стандартам, а также способствуют обмену знаниями и опытом между участниками команды.

Анализ архитектуры (Architecture Analysis) — при этом виде тестирования анализируется архитектура программного обеспечения: её структура, связи между компонентами и соблюдение архитектурных принципов. Это помогает выявить проблемы, связанные с проектированием системы.

4 Описание приложения и среды разработки

CppCheck представляет собой мощное средство статического анализа кода, написанного на языках C и C++. Оно позволяет обнаруживать разнообразные дефекты, уязвимости и места для оптимизации без необходимости компиляции или запуска программы. Проверка кода может выполняться как через командную строку, так и с помощью графического интерфейса.

Ключевые возможности CppCheck

Анализатор классифицирует найденные проблемы по уровням серьёзности:

- **Error** — критические ошибки, способные вызвать сбой программы.
- **Warning** — потенциальные проблемы, которые могут привести к некорректному поведению.
- **Style** — нарушения стиля оформления, не влияющие на исполнение, но ухудшающие читаемость кода.
- **Performance** — рекомендации по повышению производительности.
- **Portability** — вопросы, связанные с переносимостью кода на разные платформы.
- **Information** — информационные сообщения.

Для проверки кода по всем категориям можно использовать следующую команду:

```
cppcheck --enable=all --inconclusive исходные_файлы
```

Чтобы включить только определённые типы проблем, применяются соответствующие ключи:

- `-enable=style` — проверка стиля кода.
- `-enable=performance` — проверка производительности.
- `-enable=portability` — проверка переносимости.
- `-enable=information` — информационные сообщения.
- `-enable=unusedFunction` — поиск неиспользуемых функций.
- `-enable=warning` — вывод предупреждений.

Форматы вывода

CppCheck поддерживает два формата представления результатов анализа:

- текстовый;
- XML.

Правила проверки

Анализатор реализует 167 правил, сгруппированных по следующим категориям:

- 64-bit portability
- Assert
- Auto Variables
- Boolean
- Boost usage
- Bounds checking
- Check function usage
- Class
- Condition
- Exception Safety
- IO using format string
- Leaks (auto variables)
- Memory leaks (address not taken)
- Memory leaks (class variables)
- Memory leaks (function variables)
- Memory leaks (struct members)
- Null pointer
- Other
- STL usage
- Sizeof

- String
- Type
- Uninitialized variables
- Unused functions
- UnusedVar
- Using postfix operators
- Vaarg

Разбор правил группы «STL usage»

В группе «STL usage» CppCheck предоставляет 13 проверок на некорректное использование функций стандартной библиотеки шаблонов. Рассмотрим два из них.

Правило `dereferencingAnErasedIterator`

- Уровень серьёзности: Error
- Категория: STL usage

В STL действия с контейнерами обычно выполняются через итераторы (специальные объекты-посредники, предоставляющие доступ к элементам и операции над ними). Однако итераторы не всегда устойчивы: если после создания итератора контейнер был изменён, итератор может стать недействительным, и его использование недопустимо. Поведение зависит от типа контейнера. Например, для `std::vector` после вставки нового элемента все итераторы становятся невалидными, а для `std::list` они остаются корректными. Это правило выявляет проблемы, связанные с использованием итераторов после модификации контейнера.

Правило `usingAutoPointer`

- Уровень серьёзности: Warning
- Категория: STL usage

STL предоставляет несколько классов для автоматического управления ресурсами: `std::unique_ptr` (уникальное владение) и `std::shared_ptr` (разделяемое владение). Оба появились в стандарте C++11. Более старый стандарт C++03 включал только `std::auto_ptr` (также реализующий уникальное владение), но этот класс признан устаревшим. Его использование может приводить к проблемам с управлением ресурсами,

поэтому считается нежелательным. Данное правило отслеживает применение `std::auto_ptr`.

Разбор правил группы «Bounds checking»

Эта группа проверяет корректность работы с массивами и строками. Рассмотрим два правила.

Правило `arrayIndexOutOfBounds`

- Уровень серьёзности: Error
- Категория: Bounds checking

Правило действует для статических массивов, а также для динамически выделенных массивов, размер которых известен на этапе компиляции. Оно проверяет, что при обращении к массиву не происходит выхода за его границы.

Правило `negativeMemoryAllocationSize`

- Уровень серьёзности: Error
- Категория: Bounds checking

Выделение динамического или статического массива с отрицательным размером является ошибочным. Компилятор может обнаружить такую проблему только если отрицательное значение задано явно в коде. Если же размер передаётся через переменную, компилятор обычно не может выявить ошибку. Однако данное правило CppCheck способно определить проблему в тех случаях, когда значение переменной можно вычислить статически.

5 Запуск CppCheck на MacOS

Работа с CppCheck на MacOS происходит в терминале. Для того, чтобы установить и работать с программой требуется следовать следующим шагам:

1. Откройте терминал.
2. Для установки воспользуйтесь командой:

   ```
   brew install cppcheck
   ```

3. Выполните анализ программы при помощи команды:

   ```
   cppcheck --enable=all --inconclusive путь_к_исходным_файлам
   ```

Для того просмотра ошибок определенного типа используйте следующие параметры:

- `enable=style` — анализ соответствия стилевым нормам кода;
- `enable=performance` — оценка производительности;
- `enable=portability` — контроль переносимости на различные платформы;
- `enable=information` — вывод информационных сообщений;
- `enable=unusedFunction` — обнаружение функций, которые нигде не используются;
- `enable=warning` — показ предупреждений.

6 Результаты запуска

После запуска анализатора Cppcheck с параметрами `-enable=all -inconclusive` для файлов `main.cpp`, `graph.cpp` и `graph.h` было обнаружено: 1 ошибка синтаксиса, 1 стилистическое предупреждение и 7 информационных сообщений о системных заголовочных файлах.

Результаты анализа представлены в таблице ниже. В таблице указаны файл и строка, в которой обнаружена проблема, важность, краткое описание и идентификатор проблемы.

Файл	Строка	Важность	Кратко	Id
graph.h	7	ошибка	Code 'classmyGraph{' is invalid C code.	syntaxError
graph.cpp	14	стиль	The function 'getNum' is never used.	unusedFunction
main.cpp	1	информация	Include file: <iostream> not found.	missingIncludeSystem
main.cpp	2	информация	Include file: <string> not found.	missingIncludeSystem
main.cpp	3	информация	Include file: <regex> not found.	missingIncludeSystem
graph.h	3	информация	Include file: <vector> not found.	missingIncludeSystem
graph.h	4	информация	Include file: <iostream> not found.	missingIncludeSystem
graph.h	5	информация	Include file: <queue> not found.	missingIncludeSystem
main.cpp	0	информация	Limiting analysis of branches. Use –check-level=exhaustive.	normalCheckLevel-MaxBranches

Описание найденных проблем

1. **Ошибка синтаксиса:**

   Сообщение: `Code 'classmyGraph{' is invalid C code.`

   Описание: Cppcheck анализировал файл `graph.h` как код на языке C, а не C++. При таком режиме анализа ключевое слово `class` не распознаётся, и инструмент воспринимает строку `class myGraph {` как единый некорректный идентификатор `classmyGraph{`. Реальной синтаксической ошибки в коде нет.

   Решение: передавать файл на анализ явно как C++ код, добавив флаг `-language=c++`, либо убедиться, что расширение файла (`.h`, `.cpp`) корректно распознаётся анализатором.

2. **Стилистическое предупреждение:**

   Сообщение: `The function 'getNum' is never used.`

   Описание: метод `getNum()` объявлен и реализован в классе `myGraph`, однако нигде не вызывается в коде программы. После упрощения программы до нахождения кратчайшего пути метод утратил своё назначение и стал «мёртвым кодом».

   Решение: удалить объявление метода `getNum()` из файла `graph.h` и его реализацию из файла `graph.cpp`.

3. **Информационные сообщения** (`missingIncludeSystem`)**:**

   Сообщение (7 случаев): `Include file: <...> not found.`

Описание: Cppcheck не находит стандартные заголовочные файлы (`<iostream>`, `<string>`, `<regex>`, `<vector>`, `<queue>`). Как указывает сам анализатор, стандартные библиотечные заголовки не обязательно предоставлять для получения корректных результатов анализа. Это не ошибки в коде, а особенность окружения запуска Cppcheck.

Решение: сообщения носят исключительно информационный характер и не требуют исправления кода. При необходимости их можно подавить флагом `-suppress=missingIncludeSystem`.

7 Исправление недостатков

Изменения:

1. **Удаление неиспользуемого метода** `getNum()` (`unusedFunction`)**:** метод был объявлен в `graph.h` и реализован в `graph.cpp`, однако нигде не вызывался. Метод удалён из обоих файлов.

   Было в `graph.h`:

   ```
       std::vector<int> bfsShortestPath(int start, int end);
       int getNum() const;
   };
   ```

   Стало в `graph.h`:

   ```
       std::vector<int> bfsShortestPath(int start, int end);
   };
   ```

   Было в `graph.cpp`:

   ```
   int myGraph::getNum() const {
       return m_num;
   }
   ```

   Стало в `graph.cpp`: реализация метода полностью удалена.

2. **Исправление ложной ошибки синтаксиса** (`syntaxError`)**:** Cppcheck анализировал `graph.h` как код на языке C, из-за чего ключевое слово `class` не распознавалось и возникала ложная ошибка. Решение — не передавать заголовочный файл явно, так как он подключается автоматически через `#include` в `.cpp` файлах.

   Было:

   ```
   cppcheck --enable=all --inconclusive main.cpp graph.cpp graph.h
   ```

   Стало:

   ```
   cppcheck --enable=all --inconclusive main.cpp graph.cpp
   ```

После исправления всех проблем Cppcheck не выявляет ни одной ошибки, стилистического предупреждения или проблемы производительности. Результат финального запуска анализатора:

```
user@MacBook-Air-user fixed-app % cppcheck --enable=all --inconclusive main.cpp
    graph.cpp
Checking main.cpp ...
main.cpp:1:2: information: Include file: <iostream> not found. [
    missingIncludeSystem]
main.cpp:2:2: information: Include file: <string> not found. [
    missingIncludeSystem]
main.cpp:3:2: information: Include file: <regex> not found. [
    missingIncludeSystem]
```

```
graph.h:3:2: information: Include file: <vector> not found. [missingIncludeSystem]
graph.h:4:2: information: Include file: <queue> not found. [missingIncludeSystem]
main.cpp:0:0: information: Limiting analysis of branches. [normalCheckLevelMaxBranches]
1/2 files checked 74% done
Checking graph.cpp ...
2/2 files checked 100% done
nofile:0:0: information: Active checkers: 183/186 [checkersReport]
```

Итоговая статистика финального сканирования:

Категория	**Количество**
Ошибки	0
Предупреждения	0
Стилистические предупреждения	0
Предупреждения переносимости	0
Проблемы с производительностью	0
Информационные сообщения (`missingIncludeSystem`)	5
Активных анализаторов	183 из 186

Заключение

В ходе выполнения лабораторной работы был проведён статический анализ кода при помощи Cppcheck, а также описана сама программа. Использование Cppcheck позволило обнаружить одну ошибку синтаксиса и одно стилистическое предупреждение. На мой взгляд, Cppcheck — это удобный инструмент, которым легко пользоваться для тестирования программного кода.

Ошибка синтаксиса оказалась ложной: анализатор обрабатывал заголовочный файл `graph.h` как код на языке C, из-за чего ключевое слово `class` не распознавалось корректно. Проблема была устранена изменением команды запуска анализатора — без явной передачи `graph.h` в аргументах. Стилистическое предупреждение об неиспользуемом методе `getNum()` было устранено путём удаления этого метода из кода, поскольку после упрощения программы он утратил своё назначение.

Важно понимать, что статическое тестирование с помощью Cppcheck позволяет автоматически выявлять проблемы, которые сложно заметить при ручной проверке. Опыт, заложенный в инструменты статического анализа, несопоставим с возможностями одного программиста. Я сделал вывод, что статическое тестирование является полезным и необходимым этапом разработки, позволяющим повысить качество кода и устранить потенциальные ошибки на ранней стадии.